Trabajo Práctico I

SQL Avanzado: tipos, restricciones, estructuras de datos y stored procedures

MATERIA	Gestión Avanzada de Datos
CARRERA	Ingeniería en Sistemas de Información
FACULTAD	UTN
ALUMNO	Manuel Riedel
MOTOR	PostgreSQL 17
BASE DE DATOS	Facultad
FECHA	1 de septiembre de 2026

Organización de la entrega

El práctico plantea tres conjuntos de tablas independientes y dos de ellos usan los mismos nombres: el Ejercicio 2 define Alumnos y Materias con una estructura, y el Ejercicio 13 define alumnos y materias con otra completamente distinta. Para que puedan convivir en una sola base, cada parte vive en su propio esquema dentro de la base Facultad.

Esquemas

- public — Ejercicios 1 a 3 y 12: Personas, Carreras, Alumnos, Materias, Correlativas y ExamenesFinales.
- techstore — Ejercicios 4 a 11: clientes, productos, pedidos y pedido_productos.
- universidad — Ejercicios 13 a 21: alumnos, materias, inscripciones y alumnos_historicos.
- integrador — Ejercicio 22: libros, socios y prestamos.

Scripts, en orden de ejecución

```
01-facultad-tipos-y-tablas.sql      Ejercicios 1 y 2
02-facultad-datos.sql               Ejercicio 3
03-techstore-dominios-y-tipos.sql   Ejercicio 4
04-techstore-tablas.sql             Ejercicio 5
05-techstore-datos.sql              Ejercicio 6
06-techstore-consultas.sql          Ejercicios 7 a 11
07-facultad-stored-procedures.sql   Ejercicio 12
08-universidad-esquema-y-datos.sql  Ejercicio 13
09-universidad-funciones.sql        Ejercicios 14 a 21
10-integrador.sql                   Ejercicio 22
11-pruebas.sql                      Pruebas de todo lo anterior
```

Los scripts son reejecutables: cada uno borra lo que va a crear antes de crearlo. Todas las salidas que aparecen en este documento son las que devolvió PostgreSQL 17 al correrlos de punta a punta sobre una base vacía.

Una salvedad sobre el Ejercicio 21

El procedimiento migrar_alumnos_antiguos hace COMMIT adentro. PostgreSQL no permite eso si el CALL se ejecuta dentro de un bloque de transacción, así que hay que invocarlo suelto, nunca entre un BEGIN y un COMMIT externos.

Parte A — Tipos, dominios, tablas y consultas

Ejercicio 1 — Base de datos y tipos

Crear la base Facultad, los tipos enumerados TSexo, TipoDeDocumento y TPeriodoMateria, y los tipos compuestos TDocumento y TDomicilio.

Se agregó un cuarto enumerado, TEstadoExamen, porque el Ejercicio 2 pide que la columna Estado de ExamenesFinales acepte sólo Desaprobado o Aprobado.

```
CREATE TYPE TSexo           AS ENUM ('masculino', 'femenino', 'otro');
CREATE TYPE TipoDeDocumento AS ENUM ('DNI', 'Pasaporte', 'Cédula');
CREATE TYPE TPeriodoMateria AS ENUM ('Anual', 'Primer Cuatrimestre', 'Segundo Cuatrimestre');

-- El enunciado no lo pide entre los enums del Ej.1, pero la tabla
-- ExamenesFinales exige el estado (Desaprobado, Aprobado): va como enum.
CREATE TYPE TEstadoExamen   AS ENUM ('Desaprobado', 'Aprobado');

CREATE TYPE TDocumento AS (
    Numero      VARCHAR(20),
    Tipo        TipoDeDocumento,
    ExpedidoPor VARCHAR(100)
);

CREATE TYPE TDomicilio AS (
    Calle        VARCHAR(100),
    Numero       INTEGER,
    Piso         VARCHAR(10),
    Departamento VARCHAR(10)
);
```

Ejercicio 2 — Creación de tablas

Las tablas usan los tipos del ejercicio anterior. Dos decisiones que conviene señalar:

- Las columnas que son clave foránea se declaran INTEGER y no SERIAL. Con SERIAL, PostgreSQL crearía una secuencia propia y un DEFAULT nextval() sobre una columna cuyo valor tiene que venir de la tabla padre.
- Correlativas lleva clave primaria compuesta (idMateriaRequerida, idMateria), tal como pide el enunciado: sin ella se puede cargar dos veces la misma correlativa.

```
CREATE TABLE Personas (
    idPersona       SERIAL       PRIMARY KEY,
    Nombres         VARCHAR(100) NOT NULL,
    Apellidos       VARCHAR(100) NOT NULL,
    Sexo            TSexo        NOT NULL,
    Documento       TDocumento   NOT NULL,
    Domicilio       TDomicilio,
    FechaNacimiento DATE         NOT NULL,
    Foto            BYTEA,
    Huella          BYTEA,
    Firma           BYTEA
);

CREATE TABLE Carreras (
    idCarrera SERIAL      PRIMARY KEY,
    Nombre    VARCHAR(50) NOT NULL,
    Titulo    VARCHAR(50) NOT NULL,
    Posgrado  BOOLEAN     NOT NULL
);

-- Las columnas que son clave foránea van INTEGER, no SERIAL:
-- SERIAL crearía una secuencia propia y un DEFAULT nextval() sobre una
-- columna cuyo valor tiene que venir de la tabla padre.
CREATE TABLE Alumnos (
    idPersona    INTEGER     NOT NULL REFERENCES Personas(idPersona),
```

```
    Legajo       VARCHAR(10) PRIMARY KEY,
    idCarrera    INTEGER     NOT NULL REFERENCES Carreras(idCarrera),
    FechaIngreso DATE        NOT NULL,
    FechaEgreso  DATE
);

CREATE TABLE Materias (
    idMateria SERIAL          PRIMARY KEY,
    idCarrera INTEGER         NOT NULL REFERENCES Carreras(idCarrera),
    Nombre    VARCHAR(100)    NOT NULL,
    Anio      INTEGER         NOT NULL CHECK (Anio BETWEEN 1 AND 5),
    Periodo   TPeriodoMateria NOT NULL,
    Costo     NUMERIC(10,2)   NOT NULL
);

CREATE TABLE Correlativas (
    idMateriaRequerida INTEGER NOT NULL REFERENCES Materias(idMateria),
    idMateria          INTEGER NOT NULL REFERENCES Materias(idMateria),
    PRIMARY KEY (idMateriaRequerida, idMateria)
);

CREATE TABLE ExamenesFinales (
    idExamen   SERIAL        PRIMARY KEY,
    idMateria  INTEGER       NOT NULL REFERENCES Materias(idMateria),
    Legajo     VARCHAR(10)   NOT NULL REFERENCES Alumnos(Legajo),
    Fecha      DATE          NOT NULL,
    Nota       NUMERIC(4,2)  NOT NULL CHECK (Nota BETWEEN 1 AND 10),
    Estado     TEstadoExamen NOT NULL,
    NumeroActa VARCHAR(20)   NOT NULL
);
```

Ejercicio 3 — Carga de datos

Se cargaron 5 personas, 2 carreras, 8 materias de la carrera de posgrado, 9 correlativas, 5 alumnos y 25 exámenes finales entre aprobados y desaprobados.

Grafo de correlativas de la Especialización en Ingeniería de Datos

```
  1 --+--> 3 --+--> 5 --> 7 --+
     |        |               +--> 8
  2 --+--> 4 --+--> 6 ---------+

  1 Fundamentos de Bases de Datos Avanzadas   5 Arquitecturas de Big Data
  2 Estadística Aplicada a la Ingeniería      6 Aprendizaje Automático
  3 Modelado y Diseño de Datos                7 Gobierno y Calidad de Datos
  4 Minería de Datos                          8 Trabajo Final de Especialización
```

La cadena 1 ? 3 ? 5 ? 7 ? 8 da cinco niveles de profundidad, por encima de los tres que pide el enunciado.

```
TRUNCATE ExamenesFinales, Correlativas, Materias, Alumnos, Carreras, Personas
    RESTART IDENTITY CASCADE;

INSERT INTO Personas (idPersona, Nombres, Apellidos, Sexo, Documento, Domicilio, FechaNacimiento) VALUES
(1, 'Manuel',   'Riedel',    'masculino',
    ROW('40123456', 'DNI', 'Registro Nacional de las Personas')::TDocumento,
    ROW('Av. Vélez Sarsfield', 1611, '3', 'B')::TDomicilio, '1997-04-18'),

(2, 'Lucía',    'Fernández', 'femenino',
    ROW('39876543', 'DNI', 'Registro Nacional de las Personas')::TDocumento,
    ROW('Bv. Chacabuco', 850, NULL, NULL)::TDomicilio, '1996-11-02'),

(3, 'Tomás',    'Aguirre',   'masculino',
    ROW('41250987', 'DNI', 'Registro Nacional de las Personas')::TDocumento,
    ROW('Rondeau', 245, '1', 'A')::TDomicilio, '1998-07-25'),

(4, 'Sofía',    'Benítez',   'femenino',
    ROW('38445120', 'DNI', 'Registro Nacional de las Personas')::TDocumento,
    ROW('Duarte Quirós', 3300, '7', 'D')::TDomicilio, '1995-02-13'),

(5, 'Ignacio',  'Moretti',   'masculino',
```

```
    ROW('AAB123456', 'Pasaporte', 'Ministerio del Interior')::TDocumento,
    NULL, '1994-09-30');

INSERT INTO Carreras (idCarrera, Nombre, Titulo, Posgrado) VALUES
(1, 'Especialización en Ingeniería de Datos', 'Especialista en Ingeniería de Datos', TRUE),
(2, 'Ingeniería en Sistemas de Información',  'Ingeniero en Sistemas de Información', FALSE);

INSERT INTO Materias (idMateria, idCarrera, Nombre, Anio, Periodo, Costo) VALUES
(1, 1, 'Fundamentos de Bases de Datos Avanzadas', 1, 'Primer Cuatrimestre',  185000.00),
(2, 1, 'Estadística Aplicada a la Ingeniería',    1, 'Primer Cuatrimestre',  160000.00),
(3, 1, 'Modelado y Diseño de Datos',              1, 'Segundo Cuatrimestre', 195000.00),
(4, 1, 'Minería de Datos',                        1, 'Segundo Cuatrimestre', 210000.00),
(5, 1, 'Arquitecturas de Big Data',               2, 'Primer Cuatrimestre',  240000.00),
(6, 1, 'Aprendizaje Automático',                  2, 'Primer Cuatrimestre',  235000.00),
(7, 1, 'Gobierno y Calidad de Datos',             2, 'Segundo Cuatrimestre', 175000.00),
(8, 1, 'Trabajo Final de Especialización',        2, 'Anual',                120000.00);

-- Grafo de correlativas (5 niveles de profundidad):
--
--   1 ?????> 3 ?????> 5 ??> 7 ???
--       ?        ?             ???> 8
--   2 ?????> 4 ?????> 6 ?????????
--
INSERT INTO Correlativas (idMateriaRequerida, idMateria) VALUES
(1, 3),
(1, 4),
(2, 4),
(3, 5),
(4, 5),
(4, 6),
(5, 7),
(6, 8),
(7, 8);

INSERT INTO Alumnos (idPersona, Legajo, idCarrera, FechaIngreso, FechaEgreso) VALUES
(1, 'ED-0001', 1, '2024-03-04', NULL),
(2, 'ED-0002', 1, '2024-03-04', NULL),
(3, 'ED-0003', 1, '2024-03-04', NULL),
(4, 'ED-0004', 1, '2023-03-06', '2025-12-19'),
(5, 'ED-0005', 1, '2025-03-03', NULL);

INSERT INTO ExamenesFinales (idExamen, idMateria, Legajo, Fecha, Nota, Estado, NumeroActa) VALUES
-- ED-0001: aprobó 1, 2, 3, 4; desaprobó 5 y después la aprobó
( 1, 1, 'ED-0001', '2024-07-25',  8.00, 'Aprobado',    'ACTA-2024-0101'),
( 2, 2, 'ED-0001', '2024-08-01',  6.50, 'Aprobado',    'ACTA-2024-0102'),
( 3, 3, 'ED-0001', '2024-12-12',  9.00, 'Aprobado',    'ACTA-2024-0233'),
( 4, 4, 'ED-0001', '2024-12-19',  7.00, 'Aprobado',    'ACTA-2024-0241'),
( 5, 5, 'ED-0001', '2025-07-24',  3.00, 'Desaprobado', 'ACTA-2025-0118'),
( 6, 5, 'ED-0001', '2025-08-14',  6.00, 'Aprobado',    'ACTA-2025-0155'),

-- ED-0002: buen promedio, va por la última
( 7, 1, 'ED-0002', '2024-07-25', 10.00, 'Aprobado',    'ACTA-2024-0101'),
( 8, 2, 'ED-0002', '2024-08-01',  9.50, 'Aprobado',    'ACTA-2024-0102'),
( 9, 3, 'ED-0002', '2024-12-12',  8.50, 'Aprobado',    'ACTA-2024-0233'),
(10, 4, 'ED-0002', '2024-12-19',  9.00, 'Aprobado',    'ACTA-2024-0241'),
(11, 5, 'ED-0002', '2025-07-24',  8.00, 'Aprobado',    'ACTA-2025-0118'),
(12, 6, 'ED-0002', '2025-07-31',  9.00, 'Aprobado',    'ACTA-2025-0126'),
(13, 7, 'ED-0002', '2025-12-11',  7.50, 'Aprobado',    'ACTA-2025-0290'),

-- ED-0003: arrancó flojo, sólo tiene las de primer año
(14, 1, 'ED-0003', '2024-07-25',  2.00, 'Desaprobado', 'ACTA-2024-0101'),
(15, 1, 'ED-0003', '2024-08-15',  5.00, 'Aprobado',    'ACTA-2024-0117'),
(16, 2, 'ED-0003', '2024-08-01',  4.00, 'Aprobado',    'ACTA-2024-0102'),
(17, 3, 'ED-0003', '2024-12-12',  3.50, 'Desaprobado', 'ACTA-2024-0233'),
(18, 3, 'ED-0003', '2025-03-13',  6.00, 'Aprobado',    'ACTA-2025-0031'),
(19, 4, 'ED-0003', '2025-03-20',  1.50, 'Desaprobado', 'ACTA-2025-0038'),

-- ED-0004: egresada, cursó la carrera completa
(20, 1, 'ED-0004', '2023-07-27',  7.00, 'Aprobado',    'ACTA-2023-0099'),
(21, 3, 'ED-0004', '2023-12-14',  8.00, 'Aprobado',    'ACTA-2023-0210'),
(22, 5, 'ED-0004', '2024-07-25',  6.50, 'Aprobado',    'ACTA-2024-0105'),
(23, 7, 'ED-0004', '2025-07-24',  9.50, 'Aprobado',    'ACTA-2025-0121'),
(24, 8, 'ED-0004', '2025-12-05', 10.00, 'Aprobado',    'ACTA-2025-0281'),

-- ED-0005: recién ingresado, un único final desaprobado
```

```
(25, 1, 'ED-0005', '2025-07-25',  3.75, 'Desaprobado', 'ACTA-2025-0119');

-- Reposicionar las secuencias, porque los id se insertaron a mano.
SELECT setval('personas_idpersona_seq',              5);
SELECT setval('carreras_idcarrera_seq',              2);
SELECT setval('materias_idmateria_seq',              8);
SELECT setval('examenesfinales_idexamen_seq',       25);
```

Ejercicio 4 — Dominios y validaciones

email_valid exige una arroba y al menos un punto después de ella. telefono_arg acepta el formato +54 351 1234567 o una cadena de dígitos sola.

```
-- Debe contener @ y al menos un punto DESPUES del @.
CREATE DOMAIN email_valid AS TEXT
    CHECK (VALUE ~ '^[^@ ]+@[^@ ]+\.[^@ ]+$');

-- Formato argentino "+54 351 1234567", o solamente numeros.
CREATE DOMAIN telefono_arg AS TEXT
    CHECK (VALUE ~ '^\+54 [0-9]{2,4} [0-9]{6,8}$' OR VALUE ~ '^[0-9]+$');

CREATE TYPE categoria_producto AS ENUM ('electronica', 'informatica', 'hogar', 'accesorios');

-- El ENUM de estado que pide el Ejercicio 5 para la tabla pedidos.
CREATE TYPE estado_pedido AS ENUM ('pendiente', 'pagado', 'enviado', 'entregado', 'cancelado');

CREATE TYPE direccion_envio AS (
    calle         TEXT,
    numero        INT,
    piso          TEXT,
    departamento  TEXT,
    codigo_postal TEXT,
    localidad     TEXT
);
```

Ejercicio 5 — Tablas de TechStore

Además de los dominios y el ENUM, cada tabla lleva los CHECK que pide el enunciado: precio mayor a cero, stock no negativo, total mayor a cero y fecha de registro no anterior al 1 de enero de 2020. La tabla pedido_productos es la intermedia que necesita el Ejercicio 10.

```
CREATE TABLE clientes (
    id             SERIAL       PRIMARY KEY,
    nombre         TEXT         NOT NULL,
    email          email_valid  NOT NULL UNIQUE,
    telefono       telefono_arg,
    fecha_registro DATE         NOT NULL CHECK (fecha_registro >= DATE '2020-01-01')
);

CREATE TABLE productos (
    id               SERIAL             PRIMARY KEY,
    nombre           TEXT               NOT NULL,
    precio           NUMERIC(12,2)      NOT NULL CHECK (precio > 0),
    stock            INTEGER            NOT NULL CHECK (stock >= 0),
    categoria        categoria_producto NOT NULL,
    etiquetas        TEXT[],
    especificaciones JSONB
);

CREATE TABLE pedidos (
    id                SERIAL          PRIMARY KEY,
    cliente_id        INTEGER         NOT NULL REFERENCES clientes(id),
    fecha             TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    estado            estado_pedido   NOT NULL DEFAULT 'pendiente',
    direccion_entrega direccion_envio NOT NULL,
    total             NUMERIC(12,2)   NOT NULL CHECK (total > 0)
);
```

```
-- Tabla intermedia que pide el Ejercicio 10 para poder hacer el JOIN.
CREATE TABLE pedido_productos (
    pedido_id   INTEGER NOT NULL REFERENCES pedidos(id),
    producto_id INTEGER NOT NULL REFERENCES productos(id),
    cantidad    INTEGER NOT NULL CHECK (cantidad > 0),
    PRIMARY KEY (pedido_id, producto_id)
);
```

Ejercicio 6 — Inserción y prueba de validaciones

```
-- 6.1 - Tres clientes
INSERT INTO clientes (nombre, email, telefono, fecha_registro) VALUES
('Ana Gómez',      'ana.gomez@gmail.com',      '+54 351 1234567', '2021-05-14'),
('Bruno Salinas',  'bruno.salinas@hotmail.com', '3512345678',     '2023-02-01'),
('Carla Duarte',   'carla.duarte@GMAIL.COM',   '+54 11 43219876', '2024-09-23');

-- 6.2 - Cinco productos (dos con varias etiquetas)
INSERT INTO productos (nombre, precio, stock, categoria, etiquetas, especificaciones) VALUES
('Samsung Galaxy S23', 950000.00, 12, 'electronica',
 ARRAY['ofertas', 'tecnologia', 'nuevo'],
 '{"marca": "Samsung", "modelo": "Galaxy S23", "peso_gr": 168, "colores": ["negro", "blanco"]}'),

('Notebook Lenovo IdeaPad 3', 780000.00, 5, 'informatica',
 ARRAY['ofertas', 'ultima-unidad'],
 '{"marca": "Lenovo", "modelo": "IdeaPad 3", "peso_gr": 1650, "colores": ["gris"]}'),

('Cafetera Express Oster', 210000.00, 20, 'hogar',
 ARRAY['nuevo'],
 '{"marca": "Oster", "modelo": "BVSTEM6603", "peso_gr": 4200, "colores": ["negro", "plata"]}'),

('Auriculares JBL Tune 520', 89000.00, 40, 'accesorios',
 ARRAY['ofertas'],
 '{"marca": "JBL", "modelo": "Tune 520BT", "peso_gr": 160, "colores": ["azul", "negro"]}'),

('Teclado Redragon K552', 65000.00, 0, 'informatica',
 ARRAY['sin-stock'],
 '{"marca": "Redragon", "modelo": "K552", "peso_gr": 980, "colores": ["negro"]}');

-- 6.4 - Tres pedidos usando ROW(...)::direccion_envio
INSERT INTO pedidos (cliente_id, fecha, estado, direccion_entrega, total) VALUES
(1, '2025-03-10 11:20:00', 'entregado',
 ROW('Av. Colón', 1250, '4', 'B', '5000', 'Córdoba')::direccion_envio, 1039000.00),
(2, '2025-06-02 16:45:00', 'pagado',
 ROW('Rondeau', 340, NULL, NULL, '5000', 'Córdoba')::direccion_envio, 780000.00),
(3, '2025-08-19 09:05:00', 'pendiente',
 ROW('Av. Rivadavia', 5600, '2', 'A', '1424', 'Buenos Aires')::direccion_envio, 299000.00);

-- Detalle de cada pedido (Ejercicio 10)
INSERT INTO pedido_productos (pedido_id, producto_id, cantidad) VALUES
(1, 1, 1),
(1, 4, 1),
(2, 2, 1),
(3, 3, 1),
(3, 4, 1);
```

Prueba de que las restricciones rechazan lo que tienen que rechazar

Los INSERT inválidos van dentro de bloques DO con EXCEPTION, para que el error quede demostrado sin abortar el script.

```
DO $$
BEGIN
    INSERT INTO clientes (nombre, email, telefono, fecha_registro)
    VALUES ('Pepe Inválido', 'pepe@gmail', '3510000000', '2024-01-01');
    RAISE NOTICE 'ERROR: el dominio email_valid dejó pasar pepe@gmail';
EXCEPTION
    WHEN check_violation THEN
        RAISE NOTICE 'OK: email_valid rechazó "pepe@gmail" (falta el punto)';
END $$;
```

```
DO $$
BEGIN
    INSERT INTO clientes (nombre, email, telefono, fecha_registro)
    VALUES ('Tel Inválido', 'tel@mail.com', 'no-es-un-telefono', '2024-01-01');
    RAISE NOTICE 'ERROR: el dominio telefono_arg dejó pasar texto suelto';
EXCEPTION
    WHEN check_violation THEN
        RAISE NOTICE 'OK: telefono_arg rechazó "no-es-un-telefono"';
END $$;

DO $$
BEGIN
    INSERT INTO clientes (nombre, email, telefono, fecha_registro)
    VALUES ('Viejo', 'viejo@mail.com', '3510000000', '2019-12-31');
    RAISE NOTICE 'ERROR: el CHECK de fecha_registro dejó pasar 2019';
EXCEPTION
    WHEN check_violation THEN
        RAISE NOTICE 'OK: fecha_registro rechazó 2019-12-31 (anterior a 2020-01-01)';
END $$;

DO $$
BEGIN
    INSERT INTO productos (nombre, precio, stock, categoria)
    VALUES ('Precio negativo', -100, 5, 'hogar');
    RAISE NOTICE 'ERROR: el CHECK de precio dejó pasar un valor negativo';
EXCEPTION
    WHEN check_violation THEN
        RAISE NOTICE 'OK: precio rechazó -100 (debe ser > 0)';
END $$;
```

Resultado

```
NOTICE:  OK: email_valid rechazó "pepe@gmail" (falta el punto)
NOTICE:  OK: telefono_arg rechazó "no-es-un-telefono"
NOTICE:  OK: fecha_registro rechazó 2019-12-31 (anterior a 2020-01-01)
NOTICE:  OK: precio rechazó -100 (debe ser > 0)
```

Ejercicio 7 — Consultas con arrays y ENUM

```
-- 7.1 - Productos con la etiqueta 'ofertas'
SELECT id, nombre, etiquetas
FROM productos
WHERE 'ofertas' = ANY (etiquetas);

-- 7.2 - Productos con mas de 2 etiquetas
SELECT id, nombre, etiquetas, array_length(etiquetas, 1) AS cantidad_etiquetas
FROM productos
WHERE array_length(etiquetas, 1) > 2;

-- 7.3 - Pedidos en estado pendiente o pagado
SELECT id, cliente_id, estado, total
FROM pedidos
WHERE estado IN ('pendiente', 'pagado');
```

Resultado

```
-- 7.1 Productos con la etiqueta ofertas
id |          nombre           |         etiquetas
----+---------------------------+---------------------------
  1 | Samsung Galaxy S23        | {ofertas,tecnologia,nuevo}
  2 | Notebook Lenovo IdeaPad 3 | {ofertas,ultima-unidad}
  4 | Auriculares JBL Tune 520  | {ofertas}
(3 rows)

-- 7.2 Productos con mas de 2 etiquetas
id |       nombre       |         etiquetas          | cantidad_etiquetas
----+--------------------+----------------------------+--------------------
  1 | Samsung Galaxy S23 | {ofertas,tecnologia,nuevo} |                  3
(1 row)
```

```
-- 7.3 Pedidos pendientes o pagados
id | cliente_id |  estado   |   total
----+------------+-----------+-----------
  2 |          2 | pagado    | 780000.00
  3 |          3 | pendiente | 299000.00
(2 rows)
```

Ejercicio 8 — Expresiones regulares

El operador ~* compara sin distinguir mayúsculas, !~ niega la coincidencia y regexp_replace con el flag g limpia todos los caracteres que no son dígitos.

```
-- 8.1 - Clientes con email de gmail.com, sin distinguir mayusculas (~*)
SELECT id, nombre, email
FROM clientes
WHERE email ~* '@gmail\.com$';

-- 8.2 - Productos cuyo nombre NO contiene numeros
SELECT id, nombre
FROM productos
WHERE nombre !~ '[0-9]';

-- 8.3 - Desafio: telefono dejando unicamente los digitos
SELECT id,
       nombre,
       telefono,
       regexp_replace(telefono, '[^0-9]', '', 'g') AS solo_digitos
FROM clientes;
```

Resultado

```
-- 8.1 Clientes con email de gmail.com
id |    nombre    |         email
----+--------------+-----------------------
  1 | Ana Gómez    | ana.gomez@gmail.com
  3 | Carla Duarte | carla.duarte@GMAIL.COM
(2 rows)

-- 8.2 Productos sin numeros en el nombre
id |         nombre
----+------------------------
  3 | Cafetera Express Oster
(1 row)

-- 8.3 Telefonos con solo digitos
id |    nombre     |    telefono     | solo_digitos
----+---------------+-----------------+--------------
  1 | Ana Gómez     | +54 351 1234567 | 543511234567
  2 | Bruno Salinas | 3512345678      | 3512345678
  3 | Carla Duarte  | +54 11 43219876 | 541143219876
(3 rows)
```

Ejercicio 9 — Consultas y actualizaciones con JSONB

-> devuelve JSON y ->> devuelve texto. El operador @> pregunta si el JSON de la izquierda contiene al de la derecha, que es como se busca un valor dentro del array colores. jsonb_set con el cuarto parámetro en TRUE crea la clave si no existía.

```
-- 9.1 - Nombre del producto y la marca como texto (->> devuelve text)
SELECT nombre,
       especificaciones ->> 'marca' AS marca
FROM productos;

-- 9.2 - Productos con el color negro dentro del array colores
SELECT id, nombre, especificaciones -> 'colores' AS colores
FROM productos
```

```
WHERE especificaciones -> 'colores' @> '"negro"';

-- 9.3 - Agregar garantia_meses = 12 a todos los productos
UPDATE productos
SET especificaciones = jsonb_set(especificaciones, '{garantia_meses}', '12', TRUE)
WHERE especificaciones IS NOT NULL;

SELECT id, nombre, especificaciones ->> 'garantia_meses' AS garantia_meses
FROM productos;

-- 9.4 - Pedido, nombre del cliente y localidad del tipo compuesto
SELECT p.id AS id_pedido,
       c.nombre AS cliente,
       (p.direccion_entrega).localidad AS localidad
FROM pedidos p
JOIN clientes c ON c.id = p.cliente_id;
```

Resultado

```
-- 9.1 Producto y marca
nombre           |  marca
-------------------------+----------
 Samsung Galaxy S23        | Samsung
 Notebook Lenovo IdeaPad 3 | Lenovo
 Cafetera Express Oster    | Oster
 Auriculares JBL Tune 520  | JBL
 Teclado Redragon K552     | Redragon
(5 rows)

-- 9.2 Productos con color negro
id |          nombre          |       colores
----+--------------------------+---------------------
  1 | Samsung Galaxy S23       | ["negro", "blanco"]
  3 | Cafetera Express Oster   | ["negro", "plata"]
  4 | Auriculares JBL Tune 520 | ["azul", "negro"]
  5 | Teclado Redragon K552    | ["negro"]
(4 rows)

-- 9.3 Agregar garantia_meses
UPDATE 5
 id |          nombre           | garantia_meses
----+---------------------------+----------------
  1 | Samsung Galaxy S23        | 12
  2 | Notebook Lenovo IdeaPad 3 | 12
  3 | Cafetera Express Oster    | 12
  4 | Auriculares JBL Tune 520  | 12
  5 | Teclado Redragon K552     | 12
(5 rows)

-- 9.4 Pedido, cliente y localidad
id_pedido |    cliente    |  localidad
-----------+---------------+-------------
         1 | Ana Gómez     | Córdoba
         2 | Bruno Salinas | Córdoba
         3 | Carla Duarte  | Buenos Aires
(3 rows)
```

Ejercicio 10 — Consulta integradora

Se interpretó "el primer producto que contiene el pedido" como el de menor producto_id dentro de ese pedido.

```
SELECT c.nombre                       AS cliente,
       c.email                        AS email,
       p.total                        AS total_pedido,
       p.estado                       AS estado_pedido,
       (p.direccion_entrega).calle    AS calle,
       (p.direccion_entrega).numero   AS numero,
       pr.especificaciones            AS especificaciones_primer_producto
FROM pedidos p
JOIN clientes c ON c.id = p.cliente_id
JOIN pedido_productos pp ON pp.pedido_id = p.id
JOIN productos pr ON pr.id = pp.producto_id
```

```
-- "El primer producto que contiene el pedido": el de menor producto_id.
WHERE pp.producto_id = (
    SELECT MIN(pp2.producto_id)
    FROM pedido_productos pp2
    WHERE pp2.pedido_id = p.id
)
ORDER BY p.id;
```

Resultado

```
-- 10 Consulta integradora
cliente    |           email           | total_pedido | estado_pedido |     calle     | numero |
                        especificaciones_primer_producto
---------------+--------------------------+--------------+---------------+---------------+--------+-----------
-----------------------------------------------------------------------------------------------------------
 Ana Gómez     | ana.gomez@gmail.com       |   1039000.00 | entregado     | Av. Colón     |   1250 | {"marca":
"Samsung", "modelo": "Galaxy S23", "colores": ["negro", "blanco"], "peso_gr": 168, "garantia_meses": 12}
 Bruno Salinas | bruno.salinas@hotmail.com |    780000.00 | pagado        | Rondeau       |    340 | {"marca":
"Lenovo", "modelo": "IdeaPad 3", "colores": ["gris"], "peso_gr": 1650, "garantia_meses": 12}
 Carla Duarte  | carla.duarte@GMAIL.COM    |    299000.00 | pendiente     | Av. Rivadavia |   5600 | {"marca":
"Oster", "modelo": "BVSTEM6603", "colores": ["negro", "plata"], "peso_gr": 4200, "garantia_meses": 12}
(3 rows)

CREATE FUNCTION
ALTER TABLE
```

Ejercicio 11 — Bonus: restricción avanzada

Un CHECK no puede consultar otra tabla. La consulta se encapsula en una función y el CHECK la invoca. Vale aclarar la limitación: la restricción se evalúa al insertar o actualizar el pedido, no si después cambia la fecha de registro del cliente.

```
CREATE OR REPLACE FUNCTION fecha_registro_cliente(p_cliente_id INT)
RETURNS DATE AS $$
    SELECT fecha_registro FROM clientes WHERE id = p_cliente_id;
$$ LANGUAGE SQL STABLE;

ALTER TABLE pedidos
    ADD CONSTRAINT ck_entregado_despues_del_registro CHECK (
        estado <> 'entregado'
        OR fecha >= fecha_registro_cliente(cliente_id)
    );

-- Prueba: el cliente 3 se registro el 2024-09-23, este pedido es anterior.
DO $$
BEGIN
    INSERT INTO pedidos (cliente_id, fecha, estado, direccion_entrega, total)
    VALUES (3, '2024-01-05 10:00:00', 'entregado',
            ROW('Falsa', 123, NULL, NULL, '1424', 'Buenos Aires')::direccion_envio, 5000);
    RAISE NOTICE 'ERROR: se permitió entregar antes de registrar al cliente';
EXCEPTION
    WHEN check_violation THEN
        RAISE NOTICE 'OK: rechazó un pedido entregado anterior al registro del cliente';
END $$;
```

Resultado

```
-- 11 Prueba del CHECK de entrega
NOTICE:  OK: rechazó un pedido entregado anterior al registro del cliente
```

Parte B — Stored procedures y funciones en PL/pgSQL

Ejercicio 12 — Stored procedures sobre Facultad

Ejercicio 12.a

Cantidad de alumnos que cursan o cursaban una carrera a una fecha dada.

```
-- 12.a) Cantidad de alumnos que cursan o cursaban una carrera
--       a una fecha dada.
CREATE OR REPLACE FUNCTION alumnos_en_carrera(p_idcarrera INTEGER, p_fecha DATE)
RETURNS INTEGER AS $$
DECLARE
    v_cantidad INTEGER;
BEGIN
    SELECT COUNT(*)
    INTO v_cantidad
    FROM Alumnos a
    WHERE a.idCarrera = p_idcarrera
      AND a.FechaIngreso <= p_fecha
      AND (a.FechaEgreso IS NULL OR a.FechaEgreso >= p_fecha);

    RETURN v_cantidad;
END;
$$ LANGUAGE plpgsql;
```

```
cantidad
----------
       5
(1 row)
```

Ejercicio 12.b

Costo total de una carrera. Si la carrera no existe corta con RAISE EXCEPTION, usando FOUND para detectarlo.

```
-- 12.b) Costo total de una carrera.
CREATE OR REPLACE FUNCTION costo_de_carrera(p_idcarrera INTEGER)
RETURNS NUMERIC AS $$
DECLARE
    v_nombre VARCHAR(50);
    v_costo  NUMERIC;
BEGIN
    SELECT c.Nombre INTO v_nombre FROM Carreras c WHERE c.idCarrera = p_idcarrera;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'No existe la carrera %', p_idcarrera;
    END IF;

    SELECT COALESCE(SUM(m.Costo), 0)
    INTO v_costo
    FROM Materias m
    WHERE m.idCarrera = p_idcarrera;

    RAISE NOTICE 'Carrera "%": costo total %', v_nombre, v_costo;
    RETURN v_costo;
END;
$$ LANGUAGE plpgsql;
```

```
NOTICE:  Carrera "Especialización en Ingeniería de Datos": costo total 1520000.00
 costo_total
-------------
  1520000.00
(1 row)
```

Ejercicio 12.c

Promedio de un alumno considerando únicamente los exámenes aprobados.

```
-- 12.c) Promedio de un alumno, SOLO exámenes aprobados.
CREATE OR REPLACE FUNCTION promedio_aprobados(p_idpersona INTEGER, p_idcarrera INTEGER)
RETURNS NUMERIC AS $$
DECLARE
    v_promedio NUMERIC;
BEGIN
    SELECT AVG(e.Nota)
    INTO v_promedio
    FROM Alumnos a
    JOIN ExamenesFinales e ON e.Legajo = a.Legajo
    WHERE a.idPersona = p_idpersona
      AND a.idCarrera = p_idcarrera
      AND e.Estado    = 'Aprobado';

    IF v_promedio IS NULL THEN
        RAISE NOTICE 'La persona % no tiene finales aprobados en la carrera %',
                     p_idpersona, p_idcarrera;
        RETURN NULL;
    END IF;

    RETURN ROUND(v_promedio, 2);
END;
$$ LANGUAGE plpgsql;
```

Ejercicio 12.d

Promedio de un alumno considerando todas las notas.

```
-- 12.d) Promedio de un alumno, TODAS las notas.
CREATE OR REPLACE FUNCTION promedio_general(p_idpersona INTEGER, p_idcarrera INTEGER)
RETURNS NUMERIC AS $$
DECLARE
    v_promedio NUMERIC;
BEGIN
    SELECT AVG(e.Nota)
    INTO v_promedio
    FROM Alumnos a
    JOIN ExamenesFinales e ON e.Legajo = a.Legajo
    WHERE a.idPersona = p_idpersona
      AND a.idCarrera = p_idcarrera;

    IF v_promedio IS NULL THEN
        RAISE NOTICE 'La persona % no rindió finales en la carrera %',
                     p_idpersona, p_idcarrera;
        RETURN NULL;
    END IF;

    RETURN ROUND(v_promedio, 2);
END;
$$ LANGUAGE plpgsql;
```

```
solo_aprobados | todas_las_notas
---------------+-----------------
          5.00 |            3.67
(1 row)
```

Ejercicio 12.e

Datos del documento de un alumno a partir del legajo. Devuelve los tres campos del tipo compuesto TDocumento.

```
-- 12.e) Datos del documento de un alumno, a partir del Legajo.
CREATE OR REPLACE FUNCTION documento_de_alumno(p_legajo VARCHAR)
RETURNS TABLE (
    legajo        VARCHAR,
    apellidos     VARCHAR,
    nombres       VARCHAR,
    tipo          TipoDeDocumento,
```

```
    numero        VARCHAR,
    expedido_por  VARCHAR
) AS $$
BEGIN
    RETURN QUERY
    SELECT a.Legajo,
           p.Apellidos,
           p.Nombres,
           (p.Documento).Tipo,
           (p.Documento).Numero,
           (p.Documento).ExpedidoPor
    FROM Alumnos a
    JOIN Personas p ON p.idPersona = a.idPersona
    WHERE a.Legajo = p_legajo;

    IF NOT FOUND THEN
        RAISE NOTICE 'No existe el alumno con legajo %', p_legajo;
    END IF;
END;
$$ LANGUAGE plpgsql;
```

```
legajo  | apellidos | nombres |   tipo    |  numero   |      expedido_por
---------+-----------+---------+-----------+-----------+------------------------
 ED-0005 | Moretti   | Ignacio | Pasaporte | AAB123456 | Ministerio del Interior
(1 row)
```

Ejercicio 12.f

Alumnos que aprobaron una materia entre dos fechas, con apellido y nombres, legajo y nota.

```
-- 12.f) Alumnos que aprobaron una materia entre dos fechas.
CREATE OR REPLACE FUNCTION aprobados_entre_fechas(
    p_idmateria INTEGER,
    p_desde     DATE,
    p_hasta     DATE
)
RETURNS TABLE (
    apellido_y_nombres TEXT,
    legajo             VARCHAR,
    nota               NUMERIC
) AS $$
BEGIN
    RETURN QUERY
    SELECT (p.Apellidos || ', ' || p.Nombres)::TEXT,
           a.Legajo,
           e.Nota
    FROM ExamenesFinales e
    JOIN Alumnos  a ON a.Legajo    = e.Legajo
    JOIN Personas p ON p.idPersona = a.idPersona
    WHERE e.idMateria = p_idmateria
      AND e.Estado    = 'Aprobado'
      AND e.Fecha BETWEEN p_desde AND p_hasta
    ORDER BY p.Apellidos, p.Nombres;
END;
$$ LANGUAGE plpgsql;
```

```
apellido_y_nombres | legajo  | nota
--------------------+---------+-------
 Aguirre, Tomás     | ED-0003 |  5.00
 Fernández, Lucía   | ED-0002 | 10.00
 Riedel, Manuel     | ED-0001 |  8.00
(3 rows)
```

Ejercicio 12.g

Materias que hay que aprobar para poder cursar una materia determinada. Las correlativas son transitivas, así que se recorre el grafo con un WHILE y dos arrays: las materias pendientes de revisar y las ya encontradas.

```
-- 12.g) Materias que hay que aprobar para poder cursar una materia.
--
--       Las correlativas son transitivas: si 5 requiere 3 y 3
```

```
--       requiere 1, para cursar 5 hacen falta 3 y 1. Se recorre el
--       grafo con un WHILE y dos arrays (pendientes / encontradas).
CREATE OR REPLACE FUNCTION materias_requeridas(p_idmateria INTEGER)
RETURNS TABLE (
    id_materia INTEGER,
    nombre     VARCHAR,
    anio       INTEGER,
    periodo    TPeriodoMateria
) AS $$
DECLARE
    v_pendientes INTEGER[] := ARRAY[p_idmateria];
    v_requeridas INTEGER[] := ARRAY[]::INTEGER[];
    v_actual     INTEGER;
    v_correlativa INTEGER;
    v_fila       RECORD;
BEGIN
    WHILE array_length(v_pendientes, 1) > 0 LOOP
        v_actual     := v_pendientes[1];
        v_pendientes := v_pendientes[2 : array_length(v_pendientes, 1)];

        FOR v_correlativa IN
            SELECT c.idMateriaRequerida
            FROM Correlativas c
            WHERE c.idMateria = v_actual
        LOOP
            IF NOT (v_correlativa = ANY (v_requeridas)) THEN
                v_requeridas := v_requeridas || v_correlativa;
                v_pendientes := v_pendientes || v_correlativa;
            END IF;
        END LOOP;
    END LOOP;

    FOR v_fila IN
        SELECT m.idMateria, m.Nombre, m.Anio, m.Periodo
        FROM Materias m
        WHERE m.idMateria = ANY (v_requeridas)
        ORDER BY m.Anio, m.idMateria
    LOOP
        id_materia := v_fila.idMateria;
        nombre     := v_fila.Nombre;
        anio       := v_fila.Anio;
        periodo    := v_fila.Periodo;
        RETURN NEXT;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

```
id_materia |                 nombre                  | anio |       periodo
------------+-----------------------------------------+------+----------------------
          1 | Fundamentos de Bases de Datos Avanzadas |    1 | Primer Cuatrimestre
          2 | Estadística Aplicada a la Ingeniería    |    1 | Primer Cuatrimestre
          3 | Modelado y Diseño de Datos              |    1 | Segundo Cuatrimestre
          4 | Minería de Datos                        |    1 | Segundo Cuatrimestre
          5 | Arquitecturas de Big Data               |    2 | Primer Cuatrimestre
          6 | Aprendizaje Automático                  |    2 | Primer Cuatrimestre
          7 | Gobierno y Calidad de Datos             |    2 | Segundo Cuatrimestre
(7 rows)
```

Ejercicio 12.h

Costo de las materias que al alumno todavía le faltan aprobar para poder cursar una materia. Se apoya en la función anterior y descuenta las que ya aprobó.

```
-- 12.h) Costo de las materias que al alumno todavía le faltan
--       aprobar para poder cursar una materia determinada.
CREATE OR REPLACE FUNCTION costo_pendiente_para_cursar(
    p_legajo    VARCHAR,
    p_idmateria INTEGER
)
RETURNS NUMERIC AS $$
DECLARE
    v_total NUMERIC := 0;
    v_fila  RECORD;
```

```
BEGIN
    FOR v_fila IN SELECT * FROM materias_requeridas(p_idmateria) LOOP
        -- Si ya la aprobó, no la tiene que volver a cursar.
        PERFORM 1
        FROM ExamenesFinales e
        WHERE e.Legajo    = p_legajo
          AND e.idMateria = v_fila.id_materia
          AND e.Estado    = 'Aprobado';

        IF NOT FOUND THEN
            v_total := v_total + (SELECT m.Costo FROM Materias m
                                  WHERE m.idMateria = v_fila.id_materia);
        END IF;
    END LOOP;

    RETURN v_total;
END;
$$ LANGUAGE plpgsql;
```

```
costo_pendiente
-----------------
       410000.00
(1 row)
```

Ejercicio 13 — Escenario base

El esquema de la universidad, en su propio esquema para no chocar con las tablas de la Parte A. A los datos del enunciado se les sumaron una alumna y dos materias más, para que las funciones de los ejercicios siguientes tengan casos que mostrar.

```
CREATE TABLE alumnos (
    id            SERIAL PRIMARY KEY,
    nombre        TEXT NOT NULL,
    email         TEXT UNIQUE NOT NULL,
    fecha_ingreso DATE NOT NULL
);

CREATE TABLE materias (
    id       SERIAL PRIMARY KEY,
    nombre   TEXT NOT NULL,
    creditos INT NOT NULL CHECK (creditos > 0)
);

CREATE TABLE inscripciones (
    alumno_id         INT REFERENCES alumnos(id) ON DELETE CASCADE,
    materia_id        INT REFERENCES materias(id) ON DELETE CASCADE,
    fecha_inscripcion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    nota              NUMERIC(4,2) CHECK (nota >= 0 AND nota <= 10),
    PRIMARY KEY (alumno_id, materia_id)
);

-- Datos de prueba del enunciado
INSERT INTO alumnos VALUES (1, 'Ana García', 'ana@mail.com', '2023-03-01');
INSERT INTO alumnos VALUES (2, 'Luis Pérez', 'luis@mail.com', '2024-01-15');

INSERT INTO materias VALUES (1, 'Matemática', 4);
INSERT INTO materias VALUES (2, 'Física', 5);

INSERT INTO inscripciones VALUES (1, 1, '2023-03-10', 8.5);
INSERT INTO inscripciones VALUES (1, 2, '2023-04-01', 6.0);

-- Un par de filas más, para que las funciones tengan qué mostrar.
INSERT INTO alumnos  VALUES (3, 'Marta Ruiz', 'marta@mail.com', '2022-08-10');
INSERT INTO materias VALUES (3, 'Química General', 3);
INSERT INTO materias VALUES (4, 'Matemática Discreta', 6);
INSERT INTO inscripciones VALUES (3, 1, '2022-08-20', 9.5);
INSERT INTO inscripciones VALUES (3, 3, '2022-09-05', 3.5);

-- Los id se insertaron a mano: hay que reposicionar las secuencias.
SELECT setval('universidad.alumnos_id_seq',  3);
```

```
SELECT setval('universidad.materias_id_seq', 4);
```

Ejercicio 14 — RETURNS TABLE y parámetros

Devuelve alumno, materia y nota de las inscripciones cuya nota llega al mínimo pedido.

```
-- Ejercicio 14 - RETURNS TABLE y parámetros
CREATE OR REPLACE FUNCTION listar_alumnos_por_nota(nota_minima NUMERIC)
RETURNS TABLE (
    alumno TEXT,
    materia TEXT,
    nota   NUMERIC
) AS $$
BEGIN
    RETURN QUERY
    SELECT a.nombre, m.nombre, i.nota
    FROM inscripciones i
    JOIN alumnos  a ON a.id = i.alumno_id
    JOIN materias m ON m.id = i.materia_id
    WHERE i.nota >= nota_minima
    ORDER BY i.nota DESC;
END;
$$ LANGUAGE plpgsql;
```

Resultado

```
alumno   |  materia   | nota
-----------+------------+------
 Marta Ruiz | Matemática | 9.50
 Ana García | Matemática | 8.50
 Ana García | Física     | 6.00
(3 rows)
```

Ejercicio 15 — Variables y FOUND

Un detalle que importa: FOUND no sirve preguntado después del AVG, porque una función de agregado siempre devuelve una fila aunque sea NULL. Por eso la existencia de inscripciones se verifica antes con PERFORM, que sí deja FOUND en un valor útil.

```
-- Ejercicio 15 - Variables y FOUND
CREATE OR REPLACE FUNCTION obtener_promedio(p_alumno_id INT)
RETURNS NUMERIC AS $$
DECLARE
    v_promedio NUMERIC;
BEGIN
    -- Con PERFORM, FOUND queda en TRUE si la consulta devolvió filas.
    -- No sirve preguntar FOUND después del AVG: una función de agregado
    -- siempre devuelve una fila, aunque sea NULL.
    PERFORM 1 FROM inscripciones WHERE alumno_id = p_alumno_id AND nota IS NOT NULL;

    IF NOT FOUND THEN
        RAISE NOTICE 'El alumno % no tiene notas cargadas', p_alumno_id;
        RETURN NULL;
    END IF;

    SELECT AVG(nota) INTO v_promedio
    FROM inscripciones
    WHERE alumno_id = p_alumno_id;

    RETURN ROUND(v_promedio, 2);
END;
$$ LANGUAGE plpgsql;
```

Resultado

```
promedio_ana
--------------
         7.25
(1 row)

NOTICE:  El alumno 2 no tiene notas cargadas
 promedio_luis
---------------

(1 row)
```

Ejercicio 16 — CASE con parámetros IN/OUT

La nota entra como parámetro IN y la clasificación sale como OUT.

```
-- Ejercicio 16 - CASE con parámetros IN / OUT
CREATE OR REPLACE FUNCTION clasificar_nota(
    p_nota           IN  NUMERIC,
    p_clasificacion  OUT TEXT
) AS $$
BEGIN
    CASE
        WHEN p_nota >= 9 THEN p_clasificacion := 'Excelente';
        WHEN p_nota >= 7 THEN p_clasificacion := 'Aprobado';
        WHEN p_nota >= 4 THEN p_clasificacion := 'Regular';
        ELSE                  p_clasificacion := 'Reprobado';
    END CASE;
END;
$$ LANGUAGE plpgsql;
```

Resultado

```
nota | clasificacion
------+---------------
  9.5 | Excelente
  7.0 | Aprobado
  5.0 | Regular
  2.0 | Reprobado
(4 rows)
```

Ejercicio 17 — Procedimiento con FOR y RAISE NOTICE

Recorre las inscripciones e imprime una línea por cada una.

```
-- Ejercicio 17 - Procedimiento con FOR y RAISE NOTICE
CREATE OR REPLACE PROCEDURE generar_reporte_inscripciones()
AS $$
DECLARE
    v_fila RECORD;
BEGIN
    FOR v_fila IN
        SELECT a.nombre AS alumno, m.nombre AS materia, i.nota
        FROM inscripciones i
        JOIN alumnos  a ON a.id = i.alumno_id
        JOIN materias m ON m.id = i.materia_id
        ORDER BY a.nombre, m.nombre
    LOOP
        RAISE NOTICE 'Alumno % se inscribió en % con nota %',
                     v_fila.alumno, v_fila.materia, v_fila.nota;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

Resultado

```
NOTICE:  Alumno Ana García se inscribió en Física con nota 6.00
```

```
NOTICE:  Alumno Ana García se inscribió en Matemática con nota 8.50
NOTICE:  Alumno Marta Ruiz se inscribió en Matemática con nota 9.50
NOTICE:  Alumno Marta Ruiz se inscribió en Química General con nota 3.50
```

Ejercicio 18 — Manejo de excepciones

Captura foreign_key_violation cuando el alumno o la materia no existen y unique_violation cuando la inscripción ya estaba cargada. En ambos casos devuelve FALSE.

```
-- Ejercicio 18 - Manejo de excepciones
CREATE OR REPLACE FUNCTION inscribir_alumno(
    p_alumno_id  INT,
    p_materia_id INT,
    p_nota       NUMERIC DEFAULT NULL
)
RETURNS BOOLEAN AS $$
BEGIN
    INSERT INTO inscripciones (alumno_id, materia_id, nota)
    VALUES (p_alumno_id, p_materia_id, p_nota);

    RETURN TRUE;

EXCEPTION
    WHEN foreign_key_violation THEN
        RAISE NOTICE 'No existe el alumno % o la materia %', p_alumno_id, p_materia_id;
        RETURN FALSE;
    WHEN unique_violation THEN
        RAISE NOTICE 'El alumno % ya está inscripto en la materia %',
                     p_alumno_id, p_materia_id;
        RETURN FALSE;
END;
$$ LANGUAGE plpgsql;
```

Resultado

```
alta_correcta
---------------
 t
(1 row)

NOTICE:  El alumno 1 ya está inscripto en la materia 1
 ya_inscripto
--------------
 f
(1 row)

NOTICE:  No existe el alumno 99 o la materia 1
 alumno_inexistente
--------------------
 f
(1 row)
```

Ejercicio 19 — SQL dinámico y EXECUTE

quote_ident escapa los identificadores y USING pasa los valores como parámetros, que es lo que evita la inyección. Acá EXECUTE no actualiza FOUND, así que la cantidad de filas afectadas se pide con GET DIAGNOSTICS.

```
-- Ejercicio 19 - SQL dinámico y EXECUTE
CREATE OR REPLACE FUNCTION actualizar_columna_segura(
    nombre_tabla   TEXT,
    nombre_columna TEXT,
    valor_nuevo    TEXT,
    id_busqueda    INT
)
RETURNS INTEGER AS $$
DECLARE
```

```
    v_filas INTEGER;
BEGIN
    EXECUTE 'UPDATE ' || quote_ident(nombre_tabla) ||
            ' SET '   || quote_ident(nombre_columna) || ' = $1' ||
            ' WHERE id = $2'
    USING valor_nuevo, id_busqueda;

    -- EXECUTE no actualiza FOUND, así que la cantidad de filas
    -- se pide con GET DIAGNOSTICS.
    GET DIAGNOSTICS v_filas = ROW_COUNT;

    IF v_filas = 1 THEN
        RETURN 1;
    ELSE
        RETURN 0;
    END IF;

EXCEPTION
    WHEN undefined_table THEN
        RAISE NOTICE 'La tabla % no existe', nombre_tabla;
        RETURN -1;
END;
$$ LANGUAGE plpgsql;
```

Resultado

```
una_fila
----------
        1
(1 row)

 ninguna_fila
--------------
            0
(1 row)

NOTICE:  La tabla tabla_que_no_existe no existe
 tabla_inexistente
-------------------
                -1
(1 row)
```

Ejercicio 20 — RETURNS SETOF y RETURN NEXT

Devuelve filas del tipo de la tabla materias, recorriendo el resultado con un FOR y emitiendo cada registro con RETURN NEXT.

```
-- Ejercicio 20 - RETURNS SETOF y RETURN NEXT
CREATE OR REPLACE FUNCTION buscar_por_texto(palabra TEXT)
RETURNS SETOF materias AS $$
DECLARE
    v_materia materias%ROWTYPE;
BEGIN
    FOR v_materia IN
        SELECT * FROM materias
        WHERE nombre ~* palabra
        ORDER BY id
    LOOP
        RETURN NEXT v_materia;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

Resultado

```
id |       nombre        | creditos
---+---------------------+----------
  1 | Matemática          |        4
  4 | Matemática Discreta |        6
```

```
(2 rows)
```

Ejercicio 21 — Transacción y COMMIT

Copia los alumnos anteriores a la fecha de corte a alumnos_historicos, los borra de alumnos y confirma con COMMIT. Al borrar el alumno, sus inscripciones se van con él por el ON DELETE CASCADE.

```
-- Ejercicio 21 - Procedimiento con transacción y COMMIT
CREATE TABLE IF NOT EXISTS alumnos_historicos (
    id           INT PRIMARY KEY,
    nombre       TEXT NOT NULL,
    email        TEXT NOT NULL,
    fecha_egreso DATE
);

CREATE OR REPLACE PROCEDURE migrar_alumnos_antiguos(fecha_corte DATE)
AS $$
DECLARE
    v_migrados INTEGER;
BEGIN
    INSERT INTO alumnos_historicos (id, nombre, email, fecha_egreso)
    SELECT id, nombre, email, CURRENT_DATE
    FROM alumnos
    WHERE fecha_ingreso < fecha_corte;

    GET DIAGNOSTICS v_migrados = ROW_COUNT;

    DELETE FROM alumnos WHERE fecha_ingreso < fecha_corte;

    COMMIT;

    RAISE NOTICE 'Se migraron % alumnos ingresados antes del %',
                 v_migrados, fecha_corte;
END;
$$ LANGUAGE plpgsql;
```

Resultado

```
id |        nombre        | fecha_ingreso
----+----------------------+---------------
  1 | Ana García (editado) | 2023-03-01
  2 | Luis Pérez           | 2024-01-15
  3 | Marta Ruiz           | 2022-08-10
(3 rows)

NOTICE:  Se migraron 2 alumnos ingresados antes del 2024-01-01
-- alumnos que quedan:
 id |   nombre   | fecha_ingreso
----+------------+---------------
  2 | Luis Pérez | 2024-01-15
(1 row)

-- alumnos_historicos:
 id |        nombre        |     email      | fecha_egreso
----+----------------------+----------------+--------------
  1 | Ana García (editado) | ana@mail.com   | 2026-09-01
  3 | Marta Ruiz           | marta@mail.com | 2026-09-01
(2 rows)
```

Ejercicio 22 — Integración de conceptos

El caso elegido son los préstamos de una biblioteca. Reúne todo lo pedido: un enumerado y un tipo compuesto, dos dominios y varios CHECK, un array de géneros, una ficha en JSONB, claves primarias y foráneas, tres funciones en PL/pgSQL con variables y control de flujo, manejo de excepciones y SQL dinámico con EXECUTE, quote_ident y USING.

Tipos, dominios y tablas

```
-- ---------- Tipos y dominios ----------
CREATE TYPE estado_prestamo AS ENUM ('prestado', 'devuelto', 'vencido');

CREATE TYPE ubicacion AS (
    sala    TEXT,
    estante INT,
    nivel   INT
);

CREATE DOMAIN isbn_valido AS TEXT
    CHECK (VALUE ~ '^[0-9]{3}-[0-9]{10}$');

CREATE DOMAIN email_socio AS TEXT
    CHECK (VALUE ~ '^[^@ ]+@[^@ ]+\.[^@ ]+$');

-- ---------- Tablas ----------
CREATE TABLE libros (
    id         SERIAL PRIMARY KEY,
    titulo     TEXT        NOT NULL,
    isbn       isbn_valido NOT NULL UNIQUE,
    generos    TEXT[]      NOT NULL,
    ficha      JSONB,
    ubicacion  ubicacion,
    ejemplares INT         NOT NULL CHECK (ejemplares >= 0)
);

CREATE TABLE socios (
    id     SERIAL      PRIMARY KEY,
    nombre TEXT        NOT NULL,
    email  email_socio NOT NULL UNIQUE,
    alta   DATE        NOT NULL
);

CREATE TABLE prestamos (
    id               SERIAL          PRIMARY KEY,
    libro_id         INT             NOT NULL REFERENCES libros(id),
    socio_id         INT             NOT NULL REFERENCES socios(id),
    estado           estado_prestamo NOT NULL DEFAULT 'prestado',
    fecha_prestamo   DATE            NOT NULL DEFAULT CURRENT_DATE,
    fecha_devolucion DATE,
    CONSTRAINT ck_devolucion_posterior
        CHECK (fecha_devolucion IS NULL OR fecha_devolucion >= fecha_prestamo)
);
```

Datos de prueba

```
INSERT INTO libros (titulo, isbn, generos, ficha, ubicacion, ejemplares) VALUES
('El Aleph', '978-9500396073', ARRAY['cuento','fantastico'],
 '{"autor": "Jorge Luis Borges", "anio": 1949, "idiomas": ["es","en"]}',
 ROW('Sala A', 3, 2)::ubicacion, 2),

('Rayuela', '978-9500397104', ARRAY['novela','vanguardia','argentina'],
 '{"autor": "Julio Cortázar", "anio": 1963, "idiomas": ["es"]}',
 ROW('Sala A', 4, 1)::ubicacion, 1),

('Fundamentos de Bases de Datos', '978-8448190851',
 ARRAY['tecnico','informatica'],
 '{"autor": "Silberschatz", "anio": 2019, "idiomas": ["es","en"], "edicion": 7}',
 ROW('Sala B', 1, 3)::ubicacion, 0);

INSERT INTO socios (nombre, email, alta) VALUES
('Ana García',  'ana@mail.com',  '2024-03-01'),
('Luis Pérez',  'luis@mail.com', '2025-01-15');
```

Función 1 — registrar un préstamo

Usa variables, IF, FOUND, RAISE EXCEPTION propio y un bloque EXCEPTION que distingue la falta de ejemplares de un socio inexistente.

```
CREATE OR REPLACE FUNCTION registrar_prestamo(p_socio_id INT, p_libro_id INT)
RETURNS BOOLEAN AS $$
DECLARE
    v_disponibles INT;
    v_titulo      TEXT;
BEGIN
    SELECT ejemplares, titulo
    INTO v_disponibles, v_titulo
    FROM libros
    WHERE id = p_libro_id;

    IF NOT FOUND THEN
        RAISE NOTICE 'No existe el libro %', p_libro_id;
        RETURN FALSE;
    END IF;

    IF v_disponibles = 0 THEN
        RAISE EXCEPTION 'Sin ejemplares disponibles de "%"', v_titulo
            USING ERRCODE = 'check_violation';
    END IF;

    INSERT INTO prestamos (libro_id, socio_id) VALUES (p_libro_id, p_socio_id);
    UPDATE libros SET ejemplares = ejemplares - 1 WHERE id = p_libro_id;

    RAISE NOTICE 'Préstamo registrado: "%" para el socio %', v_titulo, p_socio_id;
    RETURN TRUE;

EXCEPTION
    WHEN check_violation THEN
        RAISE NOTICE 'Rechazado: %', SQLERRM;
        RETURN FALSE;
    WHEN foreign_key_violation THEN
        RAISE NOTICE 'No existe el socio %', p_socio_id;
        RETURN FALSE;
END;
$$ LANGUAGE plpgsql;
```

Función 2 — búsqueda por género

Filtra por el array de géneros con ANY y lee el autor de la ficha JSONB.

```
CREATE OR REPLACE FUNCTION libros_por_genero(p_genero TEXT)
RETURNS TABLE (
    id      INT,
    titulo  TEXT,
    autor   TEXT,
    sala    TEXT,
    generos TEXT[]
) AS $$
DECLARE
    v_fila RECORD;
BEGIN
    FOR v_fila IN
        SELECT l.id, l.titulo, l.ficha ->> 'autor' AS autor,
               (l.ubicacion).sala AS sala, l.generos
        FROM libros l
        WHERE p_genero = ANY (l.generos)
        ORDER BY l.id
    LOOP
        id      := v_fila.id;
        titulo  := v_fila.titulo;
        autor   := v_fila.autor;
        sala    := v_fila.sala;
        generos := v_fila.generos;
        RETURN NEXT;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

Función 3 — SQL dinámico

```
CREATE OR REPLACE FUNCTION contar_filas_por(
```

```
    nombre_tabla   TEXT,
    nombre_columna TEXT,
    valor          TEXT
)
RETURNS INTEGER AS $$
DECLARE
    v_cantidad INTEGER;
BEGIN
    EXECUTE 'SELECT COUNT(*) FROM ' || quote_ident(nombre_tabla) ||
            ' WHERE ' || quote_ident(nombre_columna) || '::TEXT = $1'
    INTO v_cantidad
    USING valor;

    RETURN v_cantidad;

EXCEPTION
    WHEN undefined_table THEN
        RAISE NOTICE 'La tabla % no existe', nombre_tabla;
        RETURN -1;
    WHEN undefined_column THEN
        RAISE NOTICE 'La columna % no existe en %', nombre_columna, nombre_tabla;
        RETURN -1;
END;
$$ LANGUAGE plpgsql;
```

Resultados

```
id |            titulo             |      isbn      |            generos            |  sala  | ejemplares
----+-------------------------------+----------------+-------------------------------+--------+------------
  1 | El Aleph                      | 978-9500396073 | {cuento,fantastico}           | Sala A |          2
  2 | Rayuela                       | 978-9500397104 | {novela,vanguardia,argentina} | Sala A |          1
  3 | Fundamentos de Bases de Datos | 978-8448190851 | {tecnico,informatica}         | Sala B |          0
(3 rows)
```

```
NOTICE:  Préstamo registrado: "El Aleph" para el socio 1
 prestamo_ok
-------------
 t
(1 row)

NOTICE:  Rechazado: Sin ejemplares disponibles de "Fundamentos de Bases de Datos"
 sin_ejemplares
----------------
 f
(1 row)

NOTICE:  No existe el socio 99
 socio_inexistente
-------------------
 f
(1 row)

NOTICE:  No existe el libro 77
 libro_inexistente
-------------------
 f
(1 row)
```

```
id | titulo  |     autor      |  sala  |            generos
----+---------+----------------+--------+-------------------------------
  2 | Rayuela | Julio Cortázar | Sala A | {novela,vanguardia,argentina}
(1 row)
```

```
libros_sin_stock
------------------
                1
(1 row)

 socios_ana
------------
          1
(1 row)
```

```
NOTICE:  La tabla tabla_mala no existe
 tabla_inexistente
-------------------
                -1
(1 row)
```